

Trabalho Prático 2 (TP 02) – Disciplina de Sistemas Distribuídos

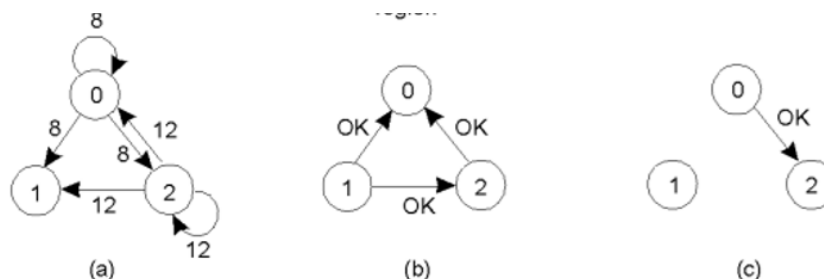
Neste trabalho o grupo de alunos (até 3 alunos) irá implementar um protocolo de sincronização distribuída. O grupo poderá implementar qualquer solução distribuída da literatura, ou seja, sem controle central de um nó do cluster, que geralmente mantém uma pilha de autorização de acesso ao recurso. ESTE TIPO DE SOLUÇÃO NÃO SERÁ VÁLIDA.

Qualquer linguagem será aceita. Não serão aceitas soluções apenas multicore.

Preliminares: Independente do protocolo a ser implementado, o seguinte cenário deve ser adotado: criar um conjunto de clientes (em containers ou VMs ou em máquinas reais mesmo) que farão acesso de escrita a um recurso R hipotético. Mínimo de 5 clientes. Cada cliente conhece apenas um elemento do cluster denominado CLUSTER DE SINCRONIZAÇÃO DE ACESSO AO RECURSO R (abreviação Cluster Sync). Cada cliente possui um ID único e a cada pedido de acesso ao recurso R, o mesmo envia o timestamp para garantir pedidos únicos. O Cluster Sync deve ser composto por, no mínimo, 5 processos encapsulados em 5 containers ou VMs ou máquinas reais. O cliente recebe apenas a mensagem COMMITTED de um elemento do Cluster Sync após o acesso a R ser concluído. Após receber uma mensagem COMMITTED, o cliente entra em estado de espera (Sleep de 1 a 5 segundos calculado aleatoriamente) e volta a pedir acesso de escrita ao recurso R. Cada cliente deve pedir entre 10 e 50 acessos de escrita ao recurso R, forçando, desta forma, a concorrência e, conseqüentemente, a validação do protocolo de exclusão mútua.

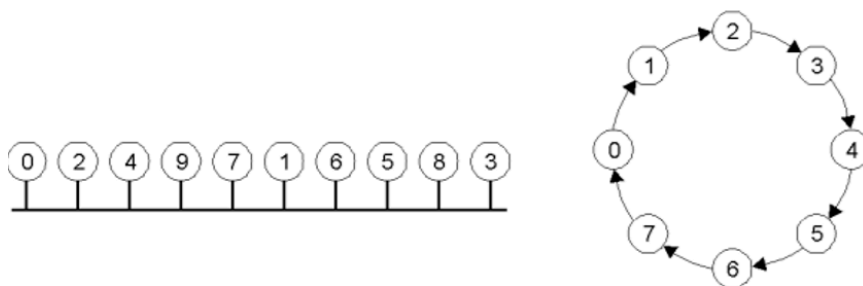
Alternativa 1 de Protocolo:

Qualquer elemento do Cluster Sync ao receber uma solicitação de um cliente obtém seu timestamp e o envia aos demais membros do Cluster Sync (incluindo ele mesmo). Também é enviado o ID (Ex. Peer1, Peer2, etc.) do elemento do Cluster Sync a cada solicitação do cliente, assim como a própria solicitação do cliente. Cada elemento do Cluster Sync avalia os timestamps recebidos e os respectivos IDs dos elementos do cluster, sabendo, com isto, sua posição temporária no acesso ao recurso R. Cada elemento do Cluster Sync responde com OK aos pares do cluster com timestamp menor do que o seu próprio. Elementos do Cluster Sync sem pedidos de cliente respondem a todos que enviaram pedido de acesso a R. O elemento do cluster com N-1 "OKs" entra na seção crítica por um tempo (Sleep de 0.2 a 1 segundo) e depois responde, tanto ao elemento do Cluster Sync com timestamp imediatamente superior ao seu, como também ao cliente. Vejamos a figura ilustrativa com 3 nós no Cluster Sync e dois nós almejando acessar R:



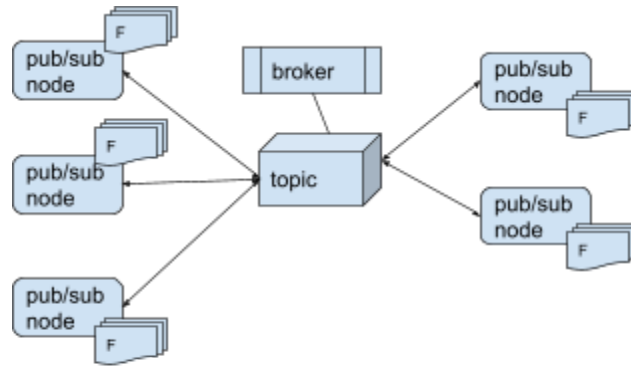
Alternativa 2 de protocolo:

Cada elemento do Cluster Sync conhece apenas um elemento de tal cluster, formando um anel. É criado um vetor com o mesmo tamanho do Cluster Sync e este passa a percorrer o anel tão logo o mesmo esteja criado. Vamos chamar tal vetor de token. Cada elemento do cluster escreve apenas na sua posição do token. Qualquer elemento do Cluster Sync ao receber uma solicitação de um cliente obtém seu timestamp, aguarda o token chegar, escreve no mesmo e o repassa. Após a inserção de um timestamp e o pedido do cliente no token, e o mesmo percorrer o anel completamente, o elemento do Cluster Sync que efetuou tal inserção já sabe se o seu timestamp é o menor gerado. Quando o token passa por um elemento do Cluster Sync e não há solicitação de um cliente, tal elemento escreve NULL na sua posição do token. Após o token retornar a cada elemento do Cluster Sync é possível saber se o mesmo entra ou não na seção crítica. Após entrar na seção crítica por um tempo (Sleep de 0.2 a 1 segundo), o elemento do Cluster Sync escreve NULL no token, responde ao cliente e repassa o token. Vejamos a figura ilustrativa:



Alternativa 3 de protocolo:

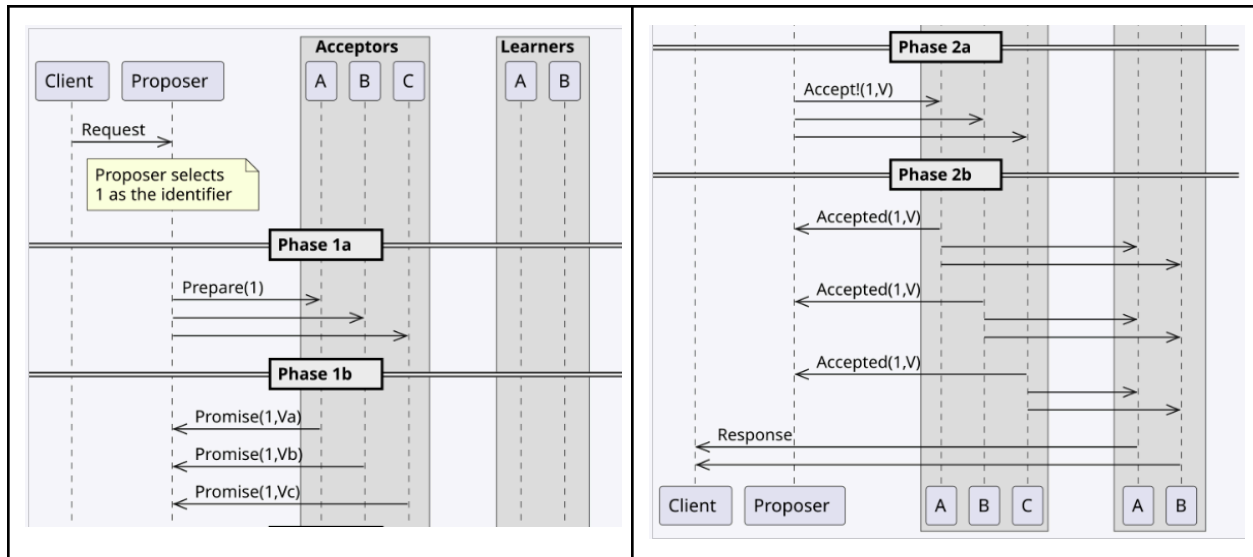
Cada elemento do Cluster Sync conhece apenas um broker pub/sub online 24x7 e tolerante a falha (Redis, RabbitMQ, Kafka ou qualquer outro). Qualquer elemento do Cluster Sync ao receber uma solicitação de um cliente envia uma mensagem de Pub a um tópico denominado R topic ou a uma fila denominada R Queue. Nesta mensagem de Pub é inserido apenas o ID do elemento do Cluster Sync, o pedido do cliente e a primitiva ACQUIRE. Cada elemento do Cluster Sync é também um consumidor, portanto recebe todas as mensagens enviadas pelo broker e o mais interessante: NA MESMA ORDEM FIFO, por exemplo. Cabe a cada elemento do Cluster Sync ficar avaliando sua fila F (PRIVADA) de mensagens do broker para saber se pode ou não entrar na seção crítica. Após entrar na seção crítica por um tempo (Sleep de 0.2 a 1 segundo), o elemento do Cluster Sync envia uma mensagem de Pub ao mesmo tópico ou fila do broker, mas neste caso contendo seu ID, o pedido do cliente e a primitiva RELEASE. Note que para entrar na seção crítica (acessar R para escrita), cada elemento do Cluster Sync deve olhar a sua ordem de ACQUIRE e também a ordem de RELEASES, assim como o pedido do cliente. Vejamos a figura ilustrativa:



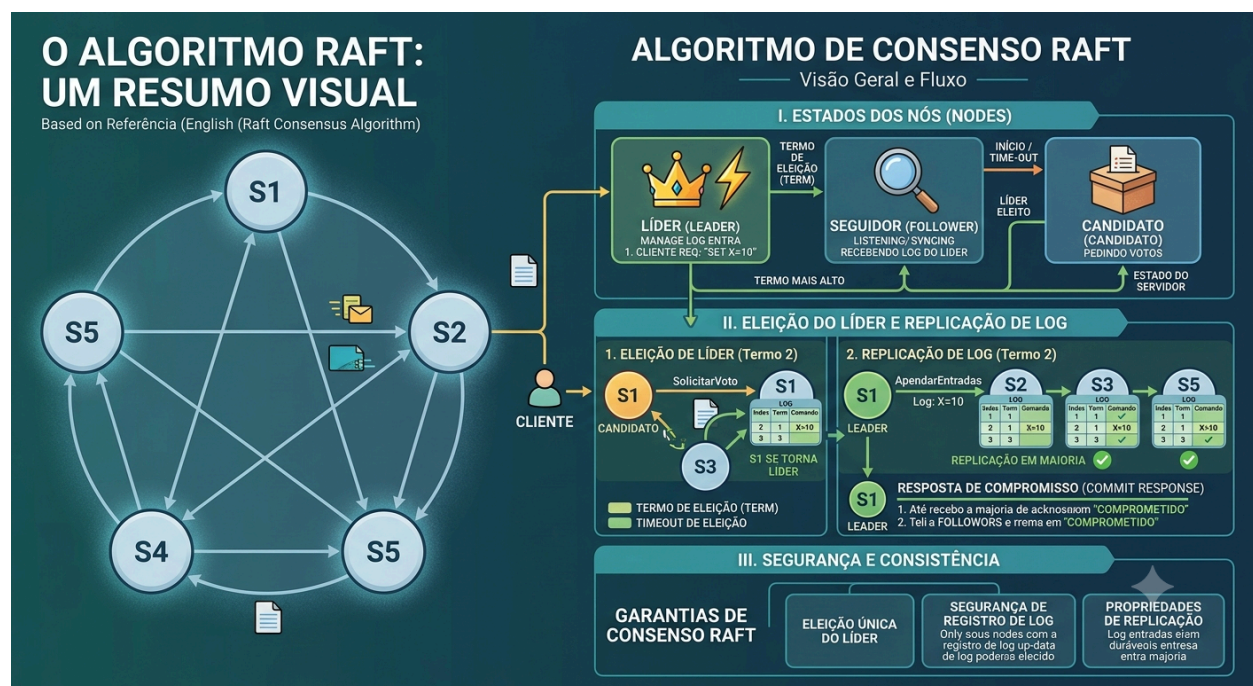
A fila F com acquies e releases possui a mesma ordem que é garantida pelo broker

Alternativa 4 de protocolo (Paxos):

Cada elemento do Cluster sync vira um “proposer”. Cada “proposer” acessa um mesmo cluster de “acceptors”, sendo 3 elementos no cluster a configuração mínima. Conforme figura a seguir, cada “proposer” submete aos “acceptors” uma proposição que deve ou não ser aceita. O “proposer” realiza commit apenas com aval dos acceptors, portanto um “acceptor” pode enviar “promise” ou “not promise”, assim como “accepted” como também “not accepted”, tudo dependendo do “acceptor” estar bloqueado ou não. Cada “promiser” mantém um valor de transação (TID) único e ajustável segundo a ordem de acesso a um determinado recurso. Tal TID é enviado pelo “promiser”, juntamente com a transação do cliente, o ID do “promiser” e seu timestamp. Cada “acceptor” mantém a lista de TIDs e o TID em uso, portanto sabe enviar “promise” ou “accepted”, assim como suas negativas, com base em suas informações privadas. Cada “acceptor” ao responder envia também o TID em uso para que o “proposer” possa corrigir, caso necessário, seu TID para numeração condizente. Cabe ao “proposer” efetivar ou não uma transação, portanto apenas com 2 ou mais mensagens “promise” para uma mesma TID que se passa para a fase “accept” na figura a seguir. Cada “acceptor” envia “accepted” ou “not accepted” para o “proposer”, mas também para um cluster de “learners” composto por 2 elementos apenas. Cabe aos “learners” responder ao cliente se foi ou não realizada a transação. O cliente re-submete, caso a transação não seja efetivada, portanto cada “learner” sempre responde ao cliente.



Alternativa 5 de Protocolo (Raft):



O algoritmo Raft de 2013. Usado pelo Kafka (Kraft) após substituição do ZooKeeper

Como funciona a Eleição no Raft

No Raft, o tempo é dividido em termos (terms) de comprimento arbitrário, numerados sequencialmente. Em qualquer momento, um nó (servidor) pode estar em um de três estados:

- **Leader (Líder):** Trata todas as requisições dos clientes e replica o log para os seguidores.

- Follower (Seguidor): Apenas responde a requisições do líder e candidatos.
- Candidate (Candidato): Estado assumido para tentar se tornar o novo líder.

A chave do Raft é o Heartbeat (batimento cardíaco) e o Timeout (tempo limite).

1. Estado Normal: O Líder envia heartbeats periódicos (mensagens vazias) para todos os Seguidores para dizer "estou vivo".
2. O Gatilho (Timeout): Se um Seguidor não recebe um heartbeat por um período de tempo (chamado election timeout, que é aleatório entre 150ms e 300ms), ele assume que o Líder morreu.
3. Candidatura:
 - O Seguidor muda seu estado para Candidate.
 - Incrementa o Termo Atual.
 - Vota em si mesmo.
 - Envia mensagens de RequestVote para todos os outros nós.
4. Votação:
 - Se o Candidato receber votos da maioria (Quorum, $N/2 + 1$), ele vira o novo Líder.
 - Imediatamente envia um heartbeat para estabelecer autoridade e impedir novas eleições.
5. Empate (Split Vote): Se muitos seguidores virarem candidatos ao mesmo tempo e ninguém conseguir a maioria, o timeout expira e uma nova eleição começa (o tempo aleatório ajuda a evitar que isso se repita indefinidamente).

Outro algoritmos -> livro Distributed.Systems.Algorithmic.Approach, cap 7.